Baseline Test and CI pipeline

Table of Contents

Introduction....2
Baseline Tests....2
test_cryptography_unit....2
test_handshake_math_correctness....3
test_end_to_end_integration....4
test_edge_cases_and_errors....5
Empty Buffer during date retrial....5
Null Inputs during Processing....5
Corrupted Data or Invalid Padding....5
Baseline Tests....5
CI Pipeline....6
Future Work....6

Introduction

This document follows the "*Baseline analysis – Initial Architecture"* document, and aims to document the initial test suites and the first implementation of our CI pipeline. In the end we will discuss the next phases and goal of the project,

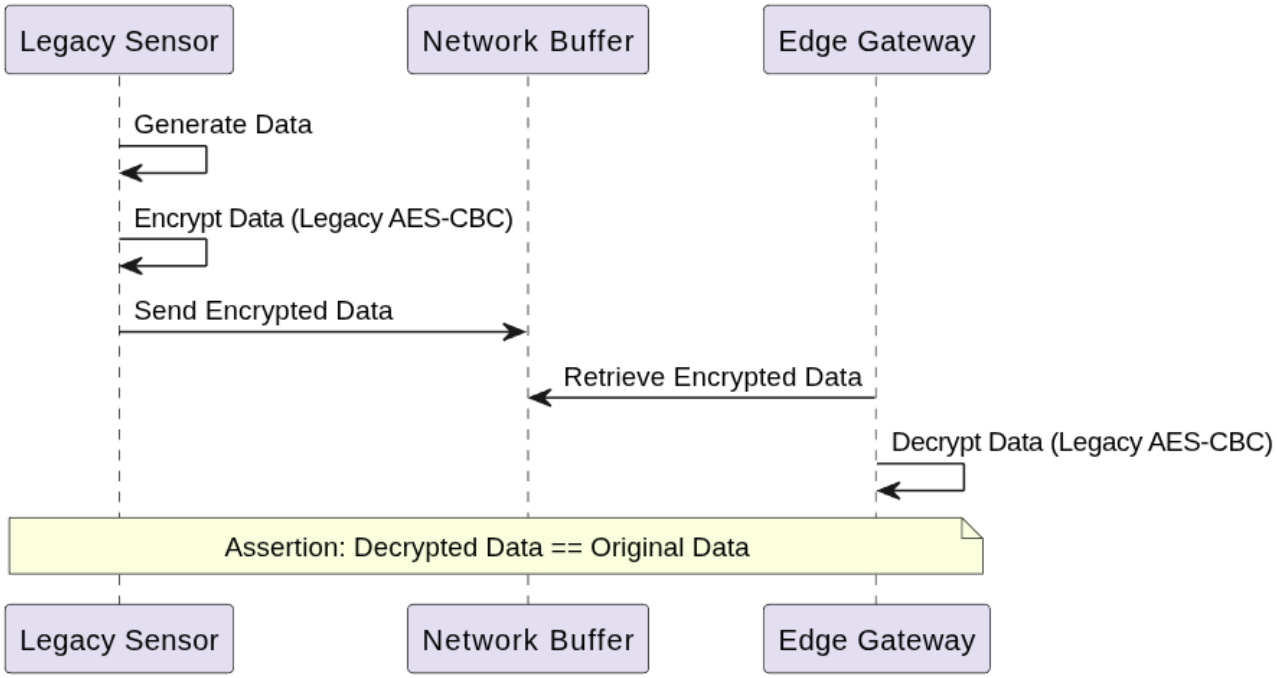
Baseline Tests

The baseline test suite is contained in *tests/test_baseline.py* and it aims to validate the core functionalities, cryptographic correctness and end-to-end integration of the system components. Moreover it sets also a performance baseline for the system that will be compared with analogous metrics at the end of the work.

Now we proceed to show and explain the test cases present in our suite.

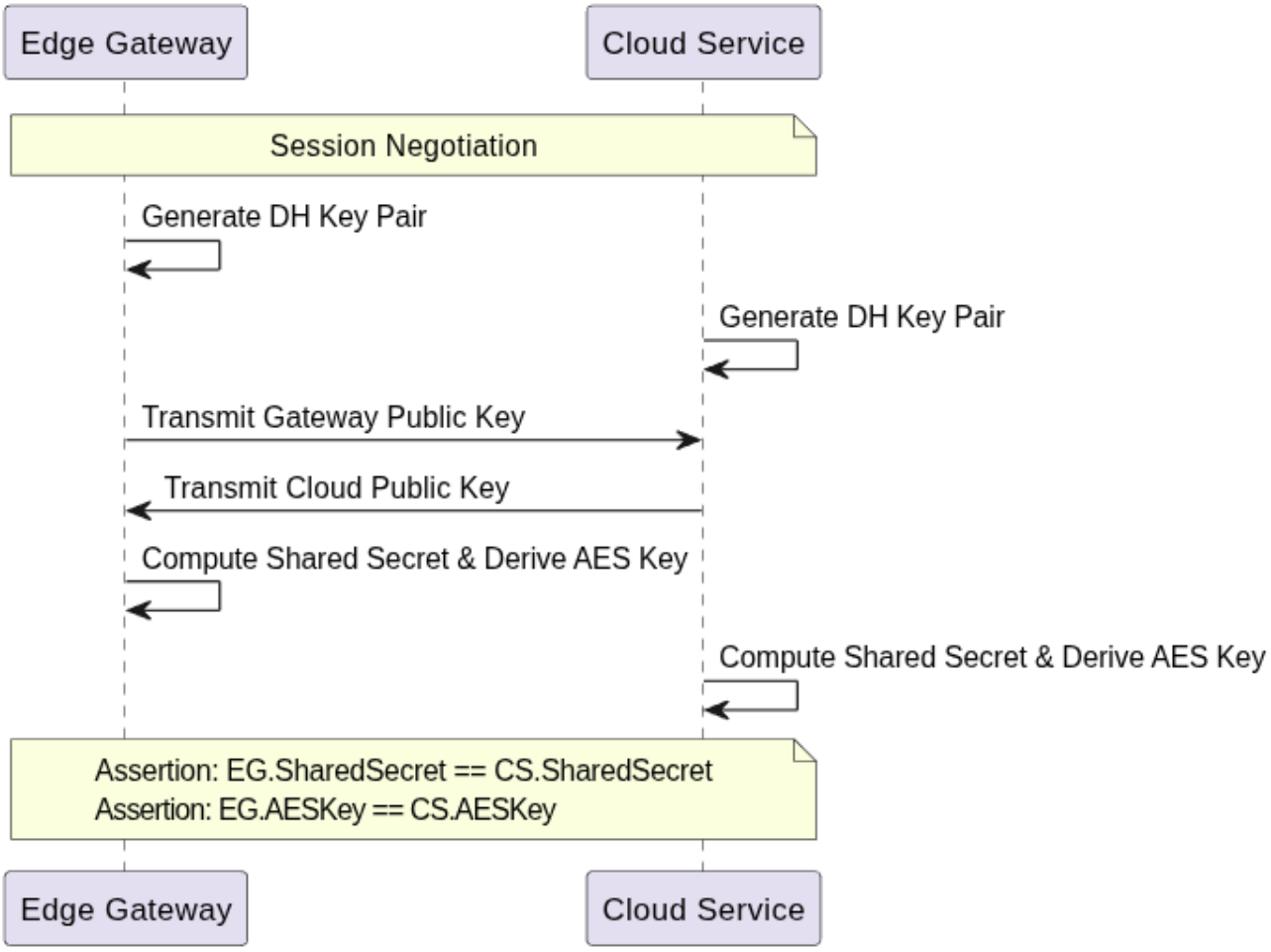
test_cryptography_unit

The purpose of this test is to verify tat the data encrypted by the Legacy Sensor using its hard-coded AES key can be successfully and deterministically decrypted by the Edge Gateway. The test works as follow:



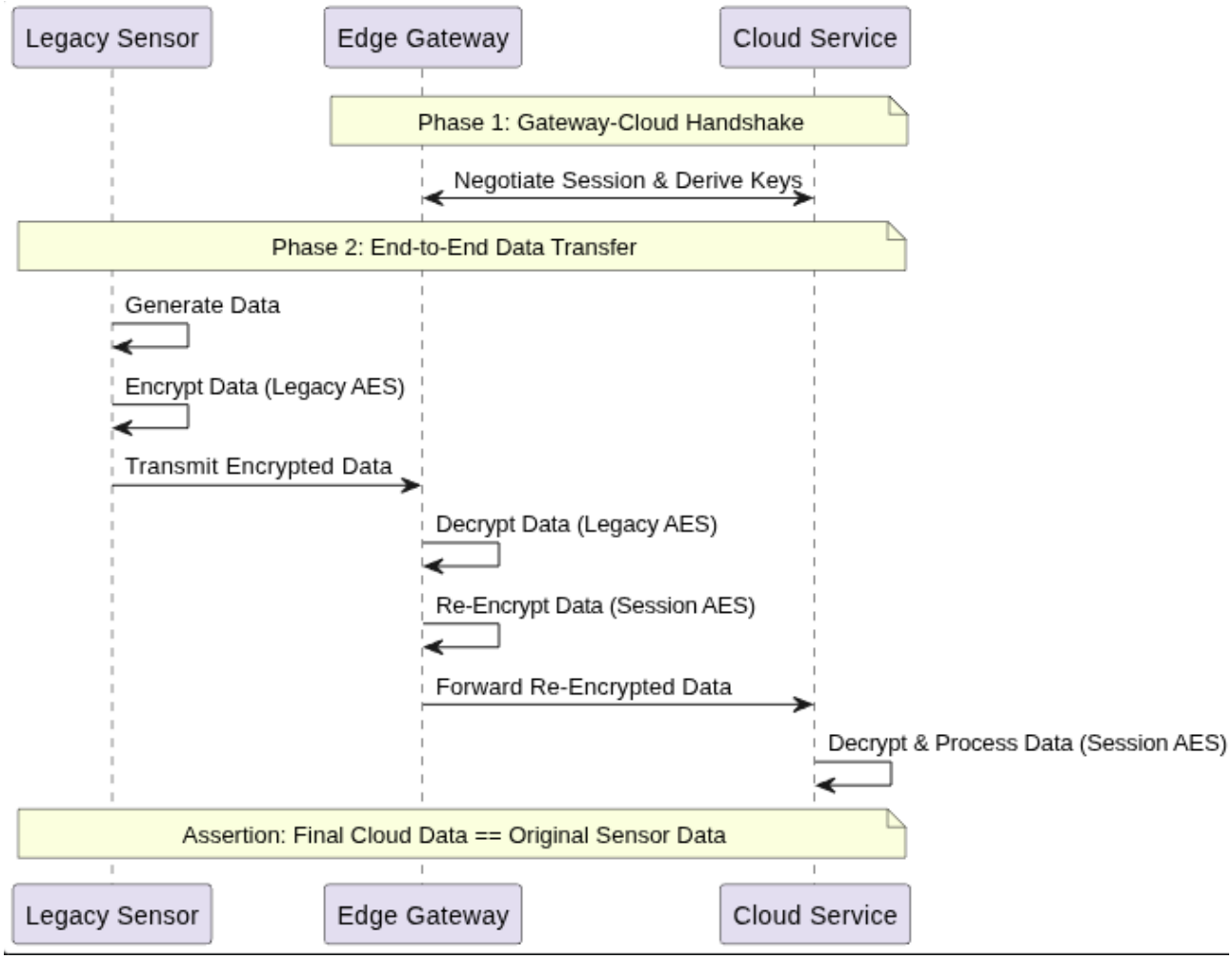
test_handshake_math_correctness

The purpose of this test is to verify the mathematical correctness of the Diffie-Hellman handshake between the Edge Gateway and the Cloud Service. The test works as follow:



test_end_to_end_integration

The purpose of this test is to verify an entire operational flow to ensure that the data correctly goes from one end of the system to the other. Moreover this test is also used to extract metrics about the system. The test work as follow:



test_edge_cases_and_errors

The purpose of this test is to verify the behavior of the system when edge cases occur and how eventual errors are handled. The exact edge cases validated by the test are the followings:

Empty Buffer during date retrial

If the Edge Gateway ties to fetch data from the receiving buffer when there are no data, it is expected that the function receiveData() return Node.

Similarly if the Cloud Service attempt to retrieve a data from its buffer when it is empty it ie expected that the function returns None.

Null Inputs during Processing

If null inputs are passed to the decryption and re-encryption primitive of the Edge Gateway it is expected that the component is capable of handling it and returns None for this type of invalid inputs.

Corrupted Data or Invalid Padding

In this part of the test we make the Device generate 32 Byte of garbage data, simulating a scenario where data is corrupted, and verify that the Cloud Service is actually capable of catching the failure and return None as expected when trying to decipher it.

Baseline Tests

The test suite documented above has been designed in order to produce metrics relative to the system and the coverage, they can also be retrieved as outputs of the CI pipeline that we will described further on.

In the following table we show the metrics recorded for what concern the baseline system:

Metric	Value
Handshake Latency (ms)	73.3503
End-to-End Latency (ms)	0.1529
Gateway Public Key Size (Bytes)	256
Cloud Public Key Size (Bytes)	256
Cipher text Size (Bytes)	48
Re-encrypted Cipher text Size (Bytes)	48

The coverage for this initial suit is 100%.

Module	Executable Statements	Missed statements	Coverage
src/cloud_service.py	39	0	100%
src/edge_gateway.py	54	0	100%
src/legacy_device.py	28	0	100%
TOTAL	121	0	100%

CI Pipeline

The CI pipeline leverages GitHub Actions to automatically run tests and report coverage. In this initial implementation the triggers are:

- push on the main branch.
- Pull requests targeting the main branch.
- Workflow dispatch that allows the pipeline to be triggered manually from the GitHub web interface in order to test the configuration.

The pipeline runs the test suite and captures coverage and performance metrics.

Future Work

The next goals of the project are:

- Implementing ML-KEM.
- Improving the CI pipelines.
- Implementing CD
- Implementing Monitoring.
- Perfective Maintenance.